Homework 10

Solutions for Questions 1 and 2 will be available after Monday, November 2, 2026. No solution will be provided for Question 3.

Use the RLT package for Questions 1 and 2: `RLT` in R or `rlt` in Python. The node-size parameter is `nmin` in R and `min_samples_leaf` in Python; both stop splitting a node when it contains at most that many observations. The candidate-predictor parameter is `mtry` in both languages. Use ordinary random forests, with reinforcement turned off.

Question 1: Bias-variance trade-off simulation

Two important tuning parameters of random forests are node size and `mtry`. We have performed some preliminary analysis in class. The goal of this question is to perform a larger simulation study for these two tuning parameters.

Consider a regression model with five jointly normally distributed predictors. Each predictor has mean 0 and variance 1, and the correlation between any two different predictors is 0.3. The response is

$$Y = X_1 + X_3 + X_5 + \varepsilon, \qquad \varepsilon \sim N(0, 1),$$

where the noise is independent of the predictors. Thus the true regression function is $f(\mathbf{x}) = x_1 + x_3 + x_5$.

a. Set up a simulation study with 100 repetitions. Generate 200 test predictor vectors once and keep them fixed. In each repetition, generate 200 new training observations and use the same training data for all tuning settings. Fit 500-tree forests, drawing 200 observations with replacement for each tree. Make the following two comparisons:

   - Fix `mtry` at 3 and vary node size over $1, 5, 10, 20, 30$.
   - Fix node size at 10 and vary `mtry` over $1, 2, 3, 4, 5$.

Leave all other fitting parameters at their defaults. Record the predictions at each test point across repetitions. For each setting, estimate the squared bias and variance at each test point, then average them over the 200 test points. Compare predictions with $f(\mathbf{x})$ when estimating bias. Produce two figures, one for each tuning parameter, showing the averaged squared bias and variance. Set a random seed and report it.

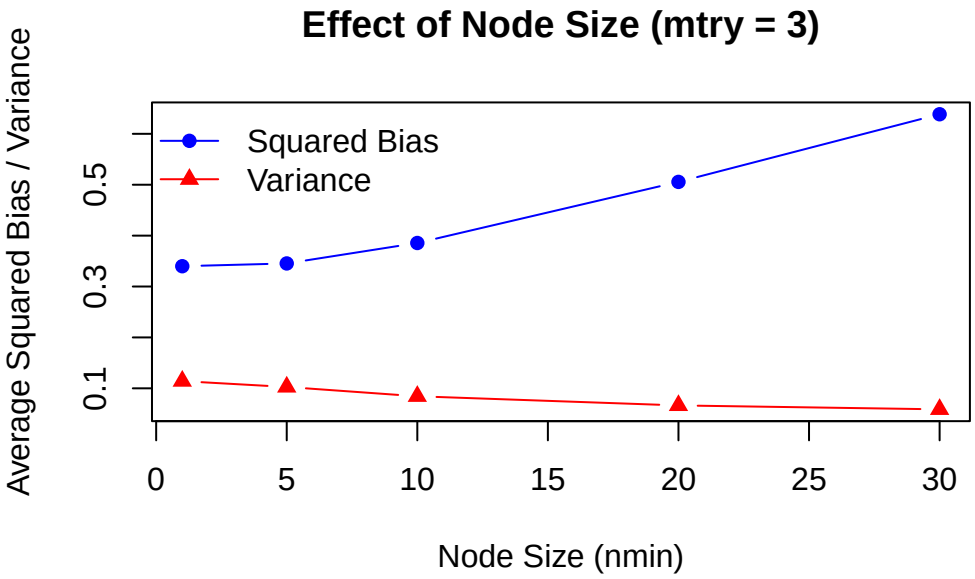
b. Discuss your results in terms of the bias-variance trade-off:

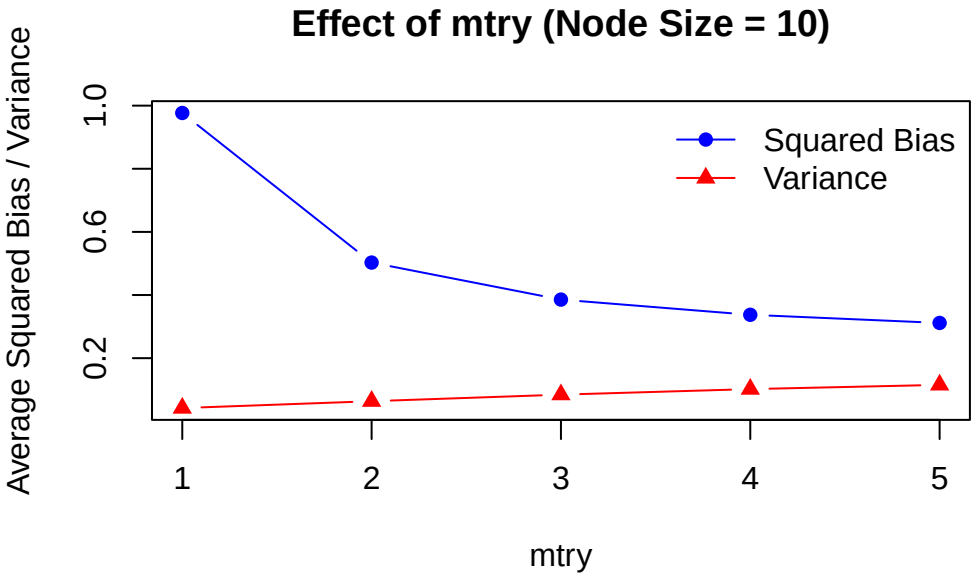
- How does a large node size, such as 30, affect the fit in this example?
- What are the benefits and drawbacks of using a large `mtry` value?
- If each tree instead sampled 100 observations with replacement, would you expect the preferred node size to shift toward smaller or larger values? Explain your reasoning. You may use an additional simulation to help, but a particular pattern is not guaranteed.

Answer (a)

The simulation uses seed 43210. The 200 test predictors are generated once and kept fixed; each of the 100 repetitions uses a new training set shared across all nine distinct tuning settings. Each forest has 500 trees and samples all 200 training observations with replacement.

Comparison	Node_size	mtry	Squared_bias	Variance
Node size	1	3	0.3399	0.1142
Node size	5	3	0.3454	0.1026
Node size	10	3	0.3855	0.0844
Node size	20	3	0.5055	0.0664
Node size	30	3	0.6385	0.0587
mtry	10	1	0.9768	0.0419
mtry	10	2	0.5028	0.0637
mtry	10	3	0.3855	0.0844
mtry	10	4	0.3374	0.1020
mtry	10	5	0.3116	0.1154


Effect of Node Size (mtry = 3)
Average Squared Bias / Variance
Squared Bias
Variance
0.1
0.3
0.5
0
5
10
15
20
25
30
Node Size (nmin)


Effect of mtry (Node Size = 10)
Average Squared Bias / Variance
Squared Bias
Variance
0.2
0.6
1.0
1
2
3
4
5
mtry

Answer (b)

With `mtry` fixed at 3, increasing the node size from 10 to 30 raises the estimated squared bias from 0.3855 to 0.6385 while reducing the estimated variance from 0.0844 to 0.0587. The larger terminal nodes smooth the fit more strongly, reducing variation but also making it harder to represent the regression function. The estimated squared-bias-plus-variance is smallest at node size 5 (0.4480), although node size 1 is close (0.4541); node sizes 20 and 30 have substantially larger sums.

With node size fixed at 10, increasing `mtry` from 1 to 5 reduces squared bias from 0.9768 to 0.3116 and increases variance from 0.0419 to 0.1154. A larger `mtry` gives each split a better chance to consider the three relevant predictors, which helps reduce bias here. The trade-off is that trees become more alike, which can increase the variability of the forest predictions. In this simulation, `mtry = 5` has the smallest estimated squared-bias-plus-variance (0.4270) among the tested settings; `mtry = 1` performs poorly because too many splits cannot consider a relevant predictor.

If each tree instead sampled 100 observations, I would expect the preferred node size to shift toward a larger value: each tree would be built from less data, and a larger minimum node size could help stabilize its predictions. This is a reason to include larger node sizes when retuning, not a guarantee of the outcome. The best setting depends on how the smaller bootstrap samples affect both the bias and the variability and correlation of the trees.

Question 2: Tuning random forests for MNIST

Use `mnist-first2000.csv.gz` from the combined ZIP. It contains 2,000 MNIST observations, with the digit label `Digit` and 784 pixel values.

- From the first 1,000 observations, keep digits 1, 6, and 7 as the training data. From the remaining 1,000 observations, keep the same digits as the test data.
- Fit PCA to the training pixels, centering but not scaling them. Use the first 20 principal components as predictors, and project the test observations using the same training means and loadings.

Complete the following tasks using RLT classification forests with 1,000 trees. For each tree, draw a bootstrap sample of the same size as the training set.

a. Fit a random forest with the default `mtry` and node-size settings. Report the out-of-bag (OOB) misclassification rate.

b. Construct a grid of `mtry` and node-size values to tune the model. Select the setting using OOB misclassification rate. Report the OOB error for each setting and the selected values. Calculate and present the selected forest's OOB permutation variable importance for the 20 principal components. Comment on your results.

c. Apply the selected model to the test data and report the misclassification rate and confusion matrix. Use the test labels only for this final assessment.

Preparation and reproducible analysis

The analysis uses seed 43210. PCA is fit only to the 327 training observations, after filtering the first 1,000 rows to digits 1, 6, and 7; the resulting training means and loadings are then applied to the 317 held-out observations. Pixels are centered but not scaled. The RLT fits use 1,000 trees, bootstrap samples the size of the training set, and have reinforcement turned off.

For each RLT fit, the OOB misclassification rate is taken from the final row of `AllError`, in the `nonPermute` column. This is the unpermuted baseline OOB error after all 1,000 trees; the predictor-named columns are permutation errors used for variable importance. I reran the default model and all 20 grid settings with the specified training split and seed, and the resulting `nonPermute` rates matched the values reported below.

Answer (a)

The RLT default settings for this training set were `mtry = 6` and `nmin = 5`. The default forest's OOB misclassification rate was **0.3200**.

Answer (b)

The grid search selected `mtry = 20` and `nmin = 20`, with an OOB misclassification rate of **0.0678**. This is a substantial reduction from the default forest's OOB error of 0.3200. Across the grid, very small `mtry` values tend to perform poorly, while several larger-`mtry` settings have lower OOB errors. The node-size pattern is less uniform, so its effect depends on `mtry`. Because the selected values are at the upper boundary of both grids, this is the best setting among the candidates tested, not evidence that these values are globally optimal; a broader grid could be explored.

The full tuning results are ordered below from lowest to highest OOB error.

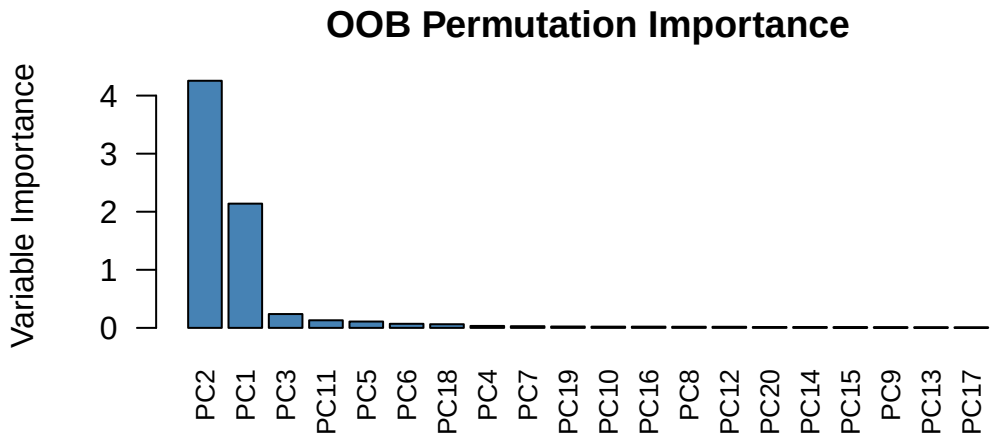
mtry	Node size (nmin)	OOB error
20	20	0.0678
15	1	0.0744
20	1	0.0847
15	10	0.1040
20	10	0.1040
20	5	0.1074

mtry	Node size (nmin)	OOB error
10	20	0.1239
10	1	0.1280
5	20	0.1327
5	10	0.1404
10	5	0.1453
5	1	0.1538
15	5	0.1667
10	10	0.2101
15	20	0.2109
2	5	0.3307
2	10	0.3628
5	5	0.3667
2	20	0.3864
2	1	0.4046

The selected forest's OOB permutation importance is concentrated in the first few principal components, especially PC2 (4.2551) and PC1 (2.1393); the next highest, PC3, is much smaller (0.2383). This suggests that these directions carry much of the information used by this fitted forest to distinguish the three digits. Importance ranks the components' contribution to this fit, not the original pixels directly, and correlated predictors can share or redistribute importance.

Principal component	OOB permutation importance
PC2	4.2551
PC1	2.1393
PC3	0.2383
PC11	0.1310
PC5	0.1092
PC6	0.0703
PC18	0.0629
PC4	0.0334
PC7	0.0289
PC19	0.0235
PC10	0.0213
PC16	0.0210
PC8	0.0192
PC12	0.0187
PC20	0.0153
PC14	0.0147
PC15	0.0130

Principal component	OOB permutation importance
PC9	0.0120
PC13	0.0107
PC17	0.0090



Answer (c)

After selecting the model using OOB error, its test-set misclassification rate was **0.0252** (8 of 317 observations), lower than the selected forest's OOB error of 0.0678. This difference can occur because OOB and test observations are different finite samples; it does not by itself establish a problem with the OOB estimate. The eight test errors were four actual 6s classified as 1 or 7, three actual 7s classified as 6, and one actual 1 classified as 7. The confusion matrix is:

Actual \ Predicted	1	6	7
1	103	0	1
6	3	102	1
7	0	3	104

The test labels were not used for PCA, tuning, or model selection.

Question 3: Planning a random-forest analysis skill

In earlier homework, some students wrote long skill files with detailed instructions, while others wrote short skills emphasizing their specific requirements or references. These approaches may perform differently depending on the AI and the task. A capable AI may already know standard ways to perform an analysis, so some instructions may be redundant, add little value, or sometimes make the analysis worse. It may be more useful to specify your personal choices for the analysis. An AI with more limited knowledge may benefit from detailed instructions that remind it of the steps to perform.

a. Write one paragraph describing how you would write a skill that directs an AI to perform a random-forest analysis and produce a report for a future dataset. Discuss how you would balance standard analysis instructions with your personal preferences, considering the AI's capabilities.

b. Provide **10 instructions** you would consider including in the skill, **ordered from the most basic instructions to the more personal preferences**. Your list may cover computation, tuning, tables, figures, interpretation, presentation, or other aspects of a random-forest report.

You only need to discuss how the skill should be written. You are not required to write a complete skill file or test it.

Answer (a)

I would write the skill to give an AI the essential safeguards for a reliable Random Forest analysis without restating every standard procedure it may already know. It should first assess whether a Random Forest is suitable for the prediction task and use a simpler baseline when that comparison is informative. The skill should also specify how to prevent data leakage, choose evaluation metrics, tune the model, and report uncertainty and limitations. To control computational cost, the AI could begin with a modest but adequate number of trees and a manageable tuning grid, check that OOB error is stable, and expand the computation when the evidence warrants it. I would make personal preferences explicit, but ask about concrete goals—such as the relative costs of false positives and false negatives, interpretability, or runtime—rather than asking users to choose an abstract bias-variance score. The AI should explain the consequences of those preferences and flag choices that materially weaken predictive performance. Similar analyses could be offered as optional references, not copied as defaults.

Answer (b)

1. Assess whether a Random Forest is appropriate for the prediction task, and consider a simpler baseline when useful.

2. Inspect the data for missing values, unusual observations, irrelevant predictors, class imbalance, and possible data leakage.
3. Keep training, validation, and test data roles separate; estimate any preprocessing steps using training data only.
4. Choose evaluation metrics that match the prediction goal and, for classification, the costs of different types of errors.
5. Fit an initial Random Forest with a reasonable number of trees and report its settings and baseline performance.
6. Tune relevant parameters such as `mtry` and node size using OOB error or another validation method appropriate to the data.
7. Check whether OOB error is stable as trees are added; expand the grid or computation when results are unstable or promising candidates remain near its boundary.
8. Use the test data only for a final assessment after model selection, and report a suitable metric and confusion matrix when applicable.
9. Present concise tables and figures, interpret variable importance cautiously, and state important limitations and computational costs.
10. Ask about the user's practical priorities—such as error costs, interpretability, and runtime—then explain how they affect model choices; offer relevant analyses as optional references without automatically copying their methods.